

1. (10%) MIPS Basics and Definitions

- A. How many registers do we have in MIPS (only worry about the version of MIPS that we have been using in class)?

32, however, we can only use a few of them. Some of them are saved for other functions.

- B. How many bits do we need to represent the above amount of registers?

32*32=1024 bits

- C. How many bits would we need to represent 16 registers?

16*32=512 bits

- D. What are the two types of memory we have discussed and how are they different?

We discussed Volatile & Non-Volatile Memory, i.e., RAM, Cache, etc. & HDDs, SSDs.

- E. Name three of the bitwise operations we used in the homeworks and explain what they do

Shift Right Logical, Shift Left Logical, bitwise NOT, AND, OR, XOR

Shifts all bits right by some number. i.e., shift 4 by 1 bit, result is 2

Shifts all bits left by some number. i.e., shift 4 by 1 bit, result is 8

NOTs a number, i.e., 4 = 0100, after NOT, it becomes :1011= 11

ANDs 2 numbers together, i.e., 1 & 3 = 0001 & 0011 = 0001

ORs 2 numbers together, i.e., 1 | 2 = 0001 | 0010 = 0011

XORs 2 numbers together i.e., 1 ^ 3 = 0001 ^ 0011 = 0010

2. (20%) Integer Representation and MIPS to Hex

- A. Convert the following 2 binary numbers to hexadecimal (hex)

a. 0101 0011 1010 1011 → **5 3 A B**

b. 1101 1000 1110 1001 → **D 8 E 9**

- B. Convert the following MIPS to Hex

a. sub \$s1, \$t2, \$a0 ANSWER: **0x01448822**

b. addiu \$s3, \$t4, 12

Following the Table, we see that we need to use the I instruction format

Our operation in Hex is 9. DECIMAL: \$s3=19, \$t4=12, 12.

9, 12, 19, 12 → *binary* → 001001, 01100, 10011, 0000000000001100

Converting to hex we get **0x2593000C**

00100101100100110000000000001100

- C. Convert the following Hex to MIPS

a. 0x004D8824 **AND \$s1 \$v0 \$t5**

b. 0xAE08000C **SW \$t0 0xC(\$s0)**

Name: Hunter Frady

3. (20%) C to MIPS. Assume that the variables f, g, h, i, j, and k are assigned to registers \$s0, \$s1, \$s2, \$s3, \$s4, \$s5 respectively. Assume that the base address of the array A is in register \$s6.

- A. What is the corresponding MIPS assembly code for the C statement
 $g = (k \mid 6) - (i + j);$ // $\mid$ stands for logical OR, & is logical AND

```
or $t0, $s5, 6  
add $t1, $s3, $s4  
sub $s1, $t0, $t1
```

- B. What is the corresponding MIPS assembly code for the C statement (Remember to move the address then use lw or sw)

```
A[j] = A[2] + (i - j);  
sub $t0, $s3, $s4  
lw $t1, 8($s6)  
add $t2, $t0, $t1  
sw $t2, $s4($s6)
```

- C. What is the corresponding MIPS assembly code for the C statement
 $i = A[i-j] + A[k-4];$

```
sub $t4, $s5, 4  
sub $t3, $s3, $s4  
lw $t1, $t3($s6)  
lw $t0, $t4($s6)  
add $s3, $t0, $t1
```

Name: Hunter Frady

4. (20%) Logical statements and branching

- A. What happens to a number when we shift left by 2?
We move all bits to the left by 2 bits, and add 2 zeroes in the 2 blank spots.
- B. What happens to a number when we shift right by 5?
We move all bits to the right by 5 bits, and add 5 zeroes in the 5 blank spots.
- C. What happens to a number when we shift right by n?
We move all bits to the right by n bits, and add n zeroes in the n blank spots.
- D. The register \$t0 has the value 0x80000001 in it (this is reset at the beginning of each part) (Answer in hex or binary is all I need) (These are unsigned)
- a. What is the value of \$t2 after the following line? srl \$t2, \$t0, 8
\$t2 = 0x00800000
- b. What is the value of \$t2 after the following lines?
sll \$t0, \$t0, 6 slti
\$t2, \$t0, 128
\$t2 = 0x00000001
- c. What is the value of \$t2 after the following lines?
sll \$t0, \$t0, 4
addi \$t1, \$zero, 16
beq \$t0, \$t1, Else
andi \$t2, \$t0, 1
j Done
Else: ori \$t2, \$t0, 1
Done:
\$t2 = 0x00000011

5. (20%) C to MIPS

A. Convert the following C code to MIPS (use registers from problem 3)

```
while(k < 5) {  
    i--;  
    j++;  
    k += 2;  
}  
while:  
slti $t0, $s5, 5  
beq $t0, $zero, end  
subi $s3, $s3, 1  
addi $s4, $s4, 1  
addi $s5, $s5, 2  
j while  
end:
```

Name: Hunter Frady

B. (For this one write the code to call it and the code for the function

```
int test_example(int a) {  
    int i = a * 4;  
    for(int j = 0; j < i; j++) {  
        printf("%d",i);  
    }  
    return a;  
}  
test_example(3);
```

```
.data  
    comma: .asciiz ", "  
.text  
j end  
test_example:  
    addi $sp, $sp, -4  
    sw $s0, 0($sp) #s0=a  
    mul $t0, $s0, 4 #int i = a * 4;  
  
for:  
    slt $t1, $t2, $t0 #t2=j #t0=i  
    beq $t1, $zero, endfor  
    addi $t2, $t2, 1  
    #print number  
    move $a0, $t0  
    li $v0, 1  
    syscall  
    # print comma between number  
    li $v0, 4  
    la $a0, comma  
    syscall  
    j for  
endfor:  
    lw $s0, 0($sp) # restore a  
    jr $ra # return  
end:  
  
li $s0, 3  
jal test_example  
#move $t0, $v0
```

Name: Hunter Frady

6. **(15%) CPI**

A. What does CPI stand for?

Cycle Per Instruction

B. Given the following tables, compute the *average* CPI for programs 1 and 2.

Instruction		Cycles					
A		1					
B		2					
C		7					
D		3					
E		1					
F		6					
Program	Instruction Count						
	A	B	C	D	E	F	
1	2	4	0	3	9	1	
2	1	3	7	1	3	2	

Type	Program 1		Program 2	
	# of instructions	CYCLES	# of instructions	CYCLES
A	2	2	1	1
B	4	8	3	6
C	0	0	7	49
D	3	9	1	3
E	9	9	3	3
F	1	6	2	12
Totals	19	34	17	74
CPI	1.789473684		4.352941176	

7. **(15%) Floating Point**

A. What is the difference between Single and Double precision?

Single precision is 32 bit and is less accurate. Double precision is 64 bit and is more accurate.

B. What is the Bias for each precision?

127 for single precision, and 1023 for double precision

C. What is the Floating Point representation of (in single precision)(only need 5 decimals of precision):

Blue is sign, green is exponent, yellow is mantissa(Cut the mantissa to 5 bits)

a. .625 → 00111111101000

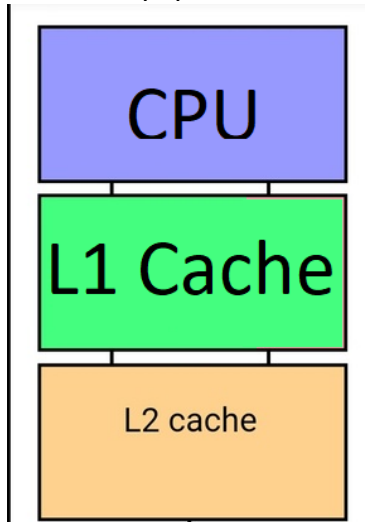
b. -.20625 → 1011111110010100

c. .04325 → 001111111001100

Name: Hunter Frady

8. **(10%) Cache and Unrolling**

- A. Draw a diagram showing how the CPU, L1, and L2 cache interact (either draw it on paper and submit a picture or use paint on your computer)



- B. Unroll the following loop 2 times and 4 times (show both new for loops):

```
#include <stdio.h>
int main(){
    for(int i = 0; i < 20; i++){
        printf("%d", i);
    }
    printf("\n");
    for(int i = 0; i < 20; i+=2){
        printf("%d", i);
        printf("%d", (i+1));
    }
    printf("\n");
    for(int i = 0; i < 20; i+=4){
        printf("%d", i);
        printf("%d", (i+1));
        printf("%d", (i+2));
        printf("%d", (i+3));
    }
    printf("\n");
}
```